

MPB Generator

Documentation complète — fonctionnement du code, UI, guide d'utilisation, troubleshooting

Sommaire

1. Présentation générale.....	5
Fonctionnalités principales.....	5
Problème résolu.....	5
2. Concepts Unity utilisés.....	6
Material.....	6
Shader.....	6
Shader Graph.....	6
Renderer.....	6
MaterialPropertyBlock.....	6
Material slot.....	6
3. Contenu et organisation du package.....	7
4. Architecture globale du tool.....	8
Architecture single-slot.....	8
Architecture multi-slot.....	8
Séparation Runtime / Editor.....	9
5. Guide d'installation.....	10
Installation comme package local.....	10
Installation simple dans Assets.....	10
Validation après installation.....	10
6. Guide rapide : MPBGenerator single-slot.....	11
Étapes.....	11
Ce qui se passe au clic Generate.....	11
Quand utiliser cette version.....	11
7. Guide rapide : MPBGeneratorList multi-slot.....	12
Étapes.....	12
Exemple.....	12
Avantages.....	12
8. Référence UI : MPBGeneratorEditor.....	13
Champs affichés.....	13
Boutons.....	13
Application automatique.....	13
9. Référence UI : MPBGeneratorListEditor.....	14
Toolbar générale.....	14
Box d'un slot.....	14
Dropdown des Materials.....	14
10. Fonctionnement du code runtime.....	15
MPBGenerator : cycle de vie.....	15
Init.....	15
GeneratePropertiesFromSourceMaterial.....	15
ApplyPropertiesToMPB.....	15
MPBGeneratorList : différences.....	16
11. Modèle de données : ShaderPropertyData et MPBSlot.....	17
ShaderPropertyData.....	17
MPBSlot.....	17
12. Fonctionnement du parser Shader Graph.....	18
Entrée et sortie.....	18
Principe.....	18
Pourquoi UnityEngine.Object ?.....	18
Limitation assumée.....	18

13. Workflow Shader Graph recommandé.....	19
Organisation conseillée.....	19
Après modification du Shader Graph.....	19
14. Gestion des types de propriétés shader.....	20
Color HDR.....	20
Range.....	20
Texture.....	20
15. Gestion des booléens et toggles.....	21
Détection actuelle.....	21
Affichage.....	21
16. Gestion des catégories.....	22
Côté UI.....	22
Catégorie Other.....	22
Ordre des catégories.....	22
17. Gestion des material slots.....	23
Dropdown de slot.....	23
Réorganisation des slots.....	23
18. Samples et exemple BloodHandSplatter.....	24
Workflow recommandé pour le sample.....	24
Contrôle de smoothness.....	24
19. Bonnes pratiques de production.....	25
Workflow recommandé pour un decal de sang.....	25
20. Performance et limites.....	26
Avantages performance.....	26
Ce que le tool ne garantit pas.....	26
Coût Editor.....	26
22. Troubleshooting détaillé.....	27
Checklist de debug rapide.....	27
Logs utiles.....	27
23. FAQ.....	28
Pourquoi ne pas utiliser directement MaterialEditor ?.....	28
Pourquoi parser le .shadergraph ?.....	28
Est-ce que le tool fonctionne avec des shaders non Shader Graph ?.....	28
MPBGeneratorList remplace-t-il MPBGenerator ?.....	28
Pourquoi les bools sont des floats ?.....	28
Les valeurs sont-elles sauvegardées dans la scène ?.....	28
Clear MPB efface-t-il Properties ?.....	28
Peut-on créer des presets ?.....	28
Pourquoi le parser est editor-only ?.....	28
Dois-je inclure les hidden properties ?.....	28
Annexe A — Glossaire.....	29
Annexe B — Snippets utiles.....	30
Auto-assign Shader Graph.....	30
Apply per material slot.....	30
Swap de slots.....	30
Boutons compatibles encodage.....	30

1. Présentation générale

MPB Generator est un outil Unity conçu pour générer, éditer et appliquer des Material Property Blocks à partir de Materials existants. L'objectif principal est de modifier des valeurs shader par Renderer ou par material slot sans dupliquer les Materials et sans devoir créer une interface custom différente pour chaque shader.

Le tool fonctionne comme un mini Material Inspector dédié aux MaterialPropertyBlocks. Il lit les propriétés exposées par le Shader d'un Material source, copie leurs valeurs dans des données sérialisées, puis applique ces valeurs dans un MaterialPropertyBlock. L'utilisateur peut ensuite ajuster les valeurs directement dans l'Inspector du GameObject.

La version actuelle est organisée comme un package Unity propre, avec un dossier Runtime, un dossier Editor, des données partagées, une documentation et un sample BloodHandSplatter. Elle contient deux workflows principaux : MPBGenerator pour un seul material slot, et MPBGeneratorList pour plusieurs material slots sur un même Renderer.

Fonctionnalités principales

- Extraction automatique des propriétés shader depuis le Material du slot sélectionné.
- Application des valeurs via MaterialPropertyBlock sans modifier le Material asset.
- Support des types Color, Vector, Float, Range, Texture et Int.
- Affichage des floats booléens sous forme de checkbox quand un attribut Toggle ou un nom toggle/bool est détecté.
- Lecture des catégories du Blackboard Shader Graph en parsant le fichier .shadergraph.
- Auto-détection du Shader Graph source depuis le shader du Material.
- Dropdown des material slots basé sur renderer.sharedMaterials.
- Version single-slot simple et version multi-slot avancée.
- Support d'un sample BloodHandSplatter pour démontrer le workflow.

Problème résolu

Dans Unity, modifier directement un Material peut soit modifier un asset partagé, soit créer une instance de Material. Sur des scènes contenant beaucoup d'objets, cela peut devenir pénible à maintenir et provoquer une multiplication involontaire des Materials.

```

Renderer
├─ sharedMaterials : Materials de base
├─ MaterialPropertyBlock global éventuel
└─ MaterialPropertyBlock par material slot possible

Material source
└─ sert de template pour générer les valeurs éditables

Shader Graph
└─ sert de source pour les catégories de l'Inspector custom
  
```

Terminologie — Dans ce document, MPB signifie MaterialPropertyBlock. MPB ne désigne jamais un Material. Un MaterialPropertyBlock contient seulement des valeurs override appliquées à un Renderer.

2. Concepts Unity utilisés

Material

Un Material est un asset Unity qui référence un Shader et stocke les valeurs par défaut des propriétés de ce Shader. Dans MPB Generator, le Material sert de modèle. Le tool lit ses valeurs, mais l'objectif est de ne pas les modifier pendant l'édition des MPB.

Shader

Le Shader expose une liste de propriétés. Le tool utilise l'API de réflexion de Unity pour parcourir ces propriétés : nom, type, flags, attributs, bornes de Range et description lisible.

```
Shader shader = sourceMaterial.shader;
int count = shader.GetPropertyCount();

for (int i = 0; i < count; i++)
{
    string propertyName = shader.GetPropertyNames(i);
    ShaderPropertyType type = shader.GetPropertyType(i);
    ShaderPropertyFlags flags = shader.GetPropertyFlags(i);
}
```

Shader Graph

Shader Graph est utilisé comme source d'organisation visuelle. Les catégories créées dans le Blackboard sont visibles dans le Material Inspector Unity, mais Unity ne fournit pas une API publique stable du type `GetPropertyCategory`. MPB Generator lit donc le fichier `.shadergraph` en Editor pour reconstruire les catégories.

Renderer

Le Renderer est le composant qui reçoit le `MaterialPropertyBlock`. Les scripts utilisent `RequireComponent(typeof(Renderer))` afin de garantir qu'un `Renderer` est présent sur le `GameObject`.

MaterialPropertyBlock

`MaterialPropertyBlock` stocke des overrides de propriétés shader pour un `Renderer`. Ces valeurs remplacent les valeurs du Material au moment du rendu, sans modifier l'asset Material lui-même.

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();

mpb.SetFloat("_BloodIntensity", 0.8f);
mpb.SetColor("_BloodColor", Color.red);

renderer.SetPropertyBlock(mpb);
```

Material slot

Un `Renderer` peut avoir plusieurs Materials dans `renderer.sharedMaterials`. Chaque entrée de ce tableau est un material slot. La version multi-slot, et maintenant la version single-slot avec dropdown, utilisent l'index de ce tableau pour cibler précisément un Material.

```
renderer.SetPropertyBlock(mpb, 0); // Element 0
renderer.SetPropertyBlock(mpb, 1); // Element 1
```

3. Contenu et organisation du package

La nouvelle organisation du package suit une structure compatible avec les packages Unity modernes. Elle sépare les scripts runtime, les scripts editor-only, la documentation et les samples.

```

com.robinfrouin.mpb-generator/
├── package.json
├── README.md
├── CHANGELOG.md
├── LICENSE.md
├── Runtime/
│   ├── MPBGenerator.cs
│   ├── MPBGeneratorList.cs
│   └── Data/
│       ├── ShaderPropertyData.cs
│       └── MPBSlot.cs
├── Editor/
│   ├── MPBGeneratorEditor.cs
│   ├── MPBGeneratorListEditor.cs
│   └── ShaderGraph/
│       └── ShaderGraphCategoryParser.cs
├── Documentation~
│   ├── MPB_Generator_Documentation_EN.pdf
│   └── MPB_Generator_Documentation_EN.docx
├── Samples~
│   └── BloodHandSplatterExample/
│       ├── Materials/
│       │   └── BloodHandSplatter.mat
│       ├── ShaderGraphs/
│       │   ├── BloodHandSplatterShaderGraph.shadergraph
│       │   └── MakeLayer.shadersubgraph
│       ├── Textures/
│       │   ├── blood_organic_grunge_noise_1024.png
│       │   └── blood_organic_grunge_noise_1024_inverted.png
│       └── Scenes/
│           └── BloodHandSplatterExample.unity

```

Chemin	Rôle
Runtime/MPBGenerator.cs	Composant single-slot. Gère un MPB pour un material slot sélectionné via dropdown.
Runtime/MPBGeneratorList.cs	Composant multi-slot. Gère une liste de MPBSlot pour plusieurs Materials du même Renderer.
Runtime/Data/ShaderPropertyData.cs	Donnée sérialisée représentant une propriété shader générée.
Runtime/Data/MPBSlot.cs	Donnée sérialisée représentant un slot MPB complet dans la version multi-slot.
Editor/MPBGeneratorEditor.cs	Custom Inspector du composant single-slot.
Editor/MPBGeneratorListEditor.cs	Custom Inspector de la version multi-slot.
Editor/ShaderGraph/ShaderGraphCategoryParser.cs	Parser editor-only du fichier .shadergraph.
Documentation~	Documentation ignorée par l'import runtime du package mais visible dans le package.
Samples~	Sample importable depuis le Package Manager.

4. Architecture globale du tool

L'architecture repose sur une chaîne simple : un Material fournit les valeurs de départ, un Shader fournit la liste des propriétés, le Shader Graph fournit les catégories, puis l'Editor dessine une interface utilisateur qui écrit dans un MaterialPropertyBlock.

```

Material du slot sélectionné
↓
Shader reflection
↓
ShaderPropertyData[]
↓
Inspector custom
↓
Modifications utilisateur
↓
MaterialPropertyBlock
↓
Renderer.SetPropertyBlock(...)

```

Architecture single-slot

```

MPBGenerator
├── MaterialIndex
├── SourceMaterial
├── ShaderGraphAsset
├── Properties : List<ShaderPropertyData>
├── GeneratePropertiesFromSourceMaterial()
├── ApplyPropertiesToMPB()
├── ClearPropertyBlock()

MPBGeneratorEditor
├── dropdown Material Slot
├── auto-sync SourceMaterial depuis renderer.sharedMaterials[MaterialIndex]
├── auto-assign ShaderGraphAsset
├── boutons Generate / Apply / Clear
├── affichage des propriétés par catégorie

```

Architecture multi-slot

```

MPBGeneratorList
├── Slots : List<MPBSlot>
│   ├── Name
│   ├── MaterialIndex
│   ├── SourceMaterial
│   ├── ShaderGraphAsset
│   ├── IncludeHiddenProperties
│   └── Properties : List<ShaderPropertyData>
├── GenerateSlot(slot)
├── ApplySlot(slot)
├── ApplyAll()
├── ClearSlot(slot)
├── ClearAll()

MPBGeneratorListEditor
├── Add MPB Slot
├── Apply All / Clear All
├── box par slot
├── dropdown Material Slot par slot
├── Generate / Apply / Clear par slot
├── réorganisation visuelle des slots

```

Séparation Runtime / Editor

Les classes runtime doivent pouvoir compiler dans un build player. Tout ce qui dépend de UnityEditor, AssetDatabase ou de la lecture du fichier .shadergraph doit rester dans le dossier Editor ou être protégé par #if UNITY_EDITOR.

```
#if UNITY_EDITOR
using UnityEditor;
#endif

#if UNITY_EDITOR
public UnityEngine.Object ShaderGraphAsset;
#endif
```


5. Guide d'installation

Installation comme package local

1. Copier le dossier `com.robinfrouin.mpb-generator` dans un dossier Packages local ou dans le dossier Packages du projet.
2. Ouvrir Unity et attendre la recompilation.
3. Vérifier que le package apparaît dans le Package Manager si `package.json` est bien présent.
4. Importer le sample `BloodHandSplatterExample` depuis le Package Manager si besoin.
5. Créer ou sélectionner un `GameObject` avec un `Renderer`.
6. Ajouter `MPBGenerator` ou `MPBGeneratorList`.

Installation simple dans Assets

7. Copier les dossiers `Runtime` et `Editor` dans `Assets/MPBGenerator`.
8. Conserver les scripts `Editor` dans un dossier nommé `Editor`.
9. Conserver les données dans `Runtime/Data` ou dans un dossier normal.
10. Attendre la compilation.
11. Ajouter un composant à un `GameObject` avec `Renderer`.

Validation après installation

Test	Résultat attendu
Ajouter <code>MPBGenerator</code>	L'Inspector affiche un dropdown <code>Material Slot</code> et les boutons <code>Generate</code> , <code>Apply</code> , <code>Clear</code> .
Ajouter <code>MPBGeneratorList</code>	L'Inspector affiche <code>Add MPB Slot</code> , <code>Apply All</code> et <code>Clear All</code> .
Cliquer <code>Generate</code>	La liste des propriétés se remplit et les catégories apparaissent si le <code>Shader Graph</code> est détecté.
Changer une valeur	Le <code>Renderer</code> se met à jour dans la <code>Scene View</code> ou <code>Game View</code> .
Clear MPB	Les overrides sont supprimés, sans effacer les données sérialisées.

6. Guide rapide : MPBGenerator single-slot

MPBGenerator est la version simple. Elle cible un seul material slot à la fois, mais elle utilise désormais un dropdown comme la version multi-slot pour éviter que l'utilisateur assigne manuellement un Material incohérent.

Étapes

12. Sélectionner un GameObject avec un Renderer.
13. Ajouter le composant MPBGenerator.
14. Choisir le Material Slot dans le dropdown : Element 0, Element 1, etc.
15. Vérifier que SourceMaterial se synchronise automatiquement avec le Material du slot choisi.
16. Vérifier que ShaderGraphAsset est auto-assigné. S'il reste vide, l'assigner manuellement.
17. Cliquer sur Refresh/Generate From Material.
18. Modifier les propriétés générées.
19. Cliquer sur Apply MPB si nécessaire.
20. Cliquer sur Clear MPB pour supprimer l'override du slot.

Ce qui se passe au clic Generate

```
Refresh/Generate From Material
↓
AutoAssignShaderGraphAsset()
↓
GeneratePropertiesFromSourceMaterial()
↓
ShaderGraphCategoryParser.GetPropertyCategories()
↓
shader.GetPropertyCount()
↓
création de ShaderPropertyData
↓
ApplyPropertiesToMPB()
↓
renderer.SetPropertyBlock(mpb, MaterialIndex)
```

Quand utiliser cette version

- Un seul Material à contrôler sur le Renderer.
- Un decal, un impact, un objet interactif ou un prop simple.
- Un outil de test rapide pour un Shader Graph.
- Un besoin de workflow léger sans liste de slots.

7. Guide rapide : MPBGeneratorList multi-slot

MPBGeneratorList est la version avancée. Elle contient une liste de MPBSlot. Chaque slot possède son MaterialIndex, son Material source, son Shader Graph et ses propres propriétés générées.

Étapes

21. Sélectionner un GameObject avec un Renderer contenant un ou plusieurs Materials.
22. Ajouter MPBGeneratorList.
23. Cliquer sur Add MPB Slot.
24. Dans la box du slot, choisir un Material Slot dans le dropdown.
25. Cliquer sur Generate This Slot.
26. Modifier les valeurs générées dans ce slot.
27. Cliquer sur Apply This Slot pour appliquer uniquement ce slot.
28. Répéter pour les autres Materials.
29. Utiliser Apply All pour appliquer tous les slots.
30. Utiliser Clear This Slot ou Clear All pour retirer les overrides.

Exemple

```
Renderer.sharedMaterials
├── Element 0 - BodyMaterial
├── Element 1 - BloodMaterial
└── Element 2 - ArmorMaterial

MPBGeneratorList.Slots
├── MPB Slot 0 → MaterialIndex 1 → BloodMaterial
└── MPB Slot 1 → MaterialIndex 2 → ArmorMaterial
```

Avantages

- Un seul composant peut gérer plusieurs material slots.
- Chaque slot peut avoir ses propres valeurs et sa propre génération.
- Le dropdown évite les erreurs d'index.
- Les slots peuvent être réorganisés dans l'UI sans changer leur MaterialIndex.
- Le workflow reste proche du Material Inspector, mais les valeurs restent dans un MPB.

8. Référence UI : MPBGeneratorEditor

Champs affichés

Champ UI	Source de donnée	Description
Material Slot	renderer.sharedMaterials	Dropdown affichant Element i - MaterialName. Met à jour MaterialIndex et SourceMaterial.
Source Material	Material du slot	Affiché comme information, généralement synchronisé automatiquement.
Include Hidden Properties	bool	Inclut les propriétés HideInInspector pendant la génération.
Shader Graph Asset	UnityEngine.Object	Asset .shadergraph auto-détection depuis le shader du Material.

Boutons

Bouton	Action
Refresh/Generate From Material	Synchronise le Shader Graph, régénère Properties et applique le MPB.
Apply MPB	Réapplique les valeurs actuelles dans le MaterialPropertyBlock.
Clear MPB	Supprime le MaterialPropertyBlock du material slot ciblé.
Debug Categories	Affiche dans la Console les propriétés, catégories et types générés.

Application automatique

À la fin de OnInspectorGUI, l'Editor peut appeler ApplyPropertiesToMPB lorsque GUI.changed est vrai. Cela donne un feedback immédiat en Scene View. Il faut cependant éviter que des changements purement UI ou de sélection déclenchent des opérations lourdes trop souvent.

```
if (GUI.changed)
{
    EditorUtility.SetDirty(script);
    script.ApplyPropertiesToMPB();
}
```

9. Référence UI : MPBGeneratorListEditor

Toolbar générale

Bouton	Description
Add MPB Slot	Ajoute un nouveau slot ciblant par défaut le premier Material du Renderer.
Apply All	Applique tous les slots générés.
Clear All	Supprime les MPB de tous les material slots du Renderer.

Box d'un slot

Élément UI	Rôle
Name	Nom libre du slot. Sert uniquement à organiser l'Inspector.
Move Up / Move Down	Réorganise l'ordre d'affichage dans la liste sans changer MaterialIndex.
Remove	Supprime le slot de la liste.
Material Slot	Dropdown construit depuis renderer.sharedMaterials.
Shader Graph	Asset .shadergraph auto-déTECTÉ ou assigné manuellement.
Include Hidden Properties	Inclut les propriétés cachées pour ce slot seulement.
Generate This Slot	Génère les propriétés depuis le Material du slot.
Apply This Slot	Applique le MPB de ce slot à son MaterialIndex.
Clear This Slot	Supprime le MPB de ce MaterialIndex.

Dropdown des Materials

```
Material[] materials = renderer.sharedMaterials;

for (int i = 0; i < materials.Length; i++)
{
    string materialName = materials[i] != null ? materials[i].name : "None";
    options[i] = $"Element {i} - {materialName}";
}

slot.MaterialIndex = EditorGUILayout.Popup(
    "Material Slot",
    slot.MaterialIndex,
    options
);
```

10. Fonctionnement du code runtime

MPBGenerator : cycle de vie

MPBGenerator utilise `ExecuteAlways` afin de fonctionner en mode Editor. `OnEnable` et `OnValidate` appellent `Init` puis réappliquent les valeurs si elles existent. Cela permet de conserver l'apparence configurée après recompilation, changement de sélection ou reload de scène.

```
private void OnEnable()
{
    Init();
    ApplyPropertiesToMPB();
}

private void OnValidate()
{
    Init();
    ApplyPropertiesToMPB();
}
```

Init

`Init` récupère le `Renderer`, crée le `MaterialPropertyBlock` si nécessaire, et peut synchroniser le `Material` source selon le slot sélectionné. Dans la version single-slot mise à jour, le `Material` est surtout piloté par le dropdown de l'Editor.

GeneratePropertiesFromSourceMaterial

Cette méthode lit le Shader du `SourceMaterial`, récupère les catégories Shader Graph, vide la liste `Properties`, puis crée un `ShaderPropertyData` pour chaque propriété compatible.

```
Shader shader = SourceMaterial.shader;
int propertyCount = shader.GetPropertyCount();

for (int i = 0; i < propertyCount; i++)
{
    string propertyName = shader.GetProperty(i);
    ShaderPropertyType propertyType = shader.GetPropertyType(i);
    ShaderPropertyFlags flags = shader.GetPropertyFlags(i);

    // filtrage HideInInspector
    // catégorie Shader Graph
    // lecture de la valeur depuis SourceMaterial
    // ajout dans Properties
}
```

ApplyPropertiesToMPB

`ApplyPropertiesToMPB` remplit un `MaterialPropertyBlock` avec les valeurs stockées dans `Properties`. La méthode choisit `SetColor`, `SetVector`, `SetFloat`, `SetTexture` ou `SetInt` selon le `ShaderPropertyType`.

```
switch (property.Type)
{
    case ShaderPropertyType.Color:
        _mpb.SetColor(property.Name, property.ColorValue);
        break;

    case ShaderPropertyType.Float:
    case ShaderPropertyType.Range:
        _mpb.SetFloat(property.Name, property.FloatValue);
        break;

    case ShaderPropertyType.Texture:
        if (property.TextureValue != null)
            _mpb.SetTexture(property.Name, property.TextureValue);
        break;
}

_renderer.SetPropertyBlock(_mpb, MaterialIndex);
```

MPBGeneratorList : différences

MPBGeneratorList ne possède pas une seule liste Properties globale. Chaque MPBSlot contient son propre Material source, son Shader Graph et ses propriétés. ApplySlot crée un MaterialPropertyBlock local pour éviter de partager accidentellement un bloc entre plusieurs material slots.

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();  
  
// Remplir le MPB depuis slot.Properties  
  
_renderer.SetPropertyBlock(mpb, slot.MaterialIndex);
```

11. Modèle de données : ShaderPropertyData et MPBSlot

ShaderPropertyData

ShaderPropertyData est une structure de données sérialisée qui représente une propriété shader générée. Elle contient à la fois les informations nécessaires à l'UI et la valeur modifiable par l'utilisateur.

Champ	Type	Rôle
Name	string	Nom technique utilisé par SetFloat, SetColor, etc. Exemple : BloodColor.
DisplayName	string	Nom lisible récupéré via shader.GetPropertyDescription.
Category	string	Nom du groupe d'affichage issu du Shader Graph ou Other.
Type	ShaderPropertyType	Détermine le champ UI et la méthode MPB à utiliser.
Flags	ShaderPropertyFlags	Permet notamment de détecter HDR ou HideInInspector.
IsToggle	bool	Indique si un Float doit être affiché comme checkbox.
FloatValue	float	Valeur des propriétés Float et Range.
RangeMin / RangeMax	float	Bornes du slider pour les propriétés Range.
ColorValue	Color	Valeur de couleur.
VectorValue	Vector4	Valeur vectorielle.
TextureValue	Texture	Référence texture.
IntValue	int	Valeur entière.

MPBSlot

MPBSlot est utilisé par MPBGeneratorList. Il représente un bloc complet de génération et d'édition pour un Material slot du Renderer.

Champ	Type	Rôle
Name	string	Nom affiché dans la box du slot.
MaterialIndex	int	Index dans renderer.sharedMaterials.
SourceMaterial	Material	Material utilisé comme template de génération.
ShaderGraphAsset	UnityEngine.Object	Asset .shadergraph utilisé par le parser.
IncludeHiddenProperties	bool	Inclut les propriétés cachées pour ce slot.
Properties	List<ShaderPropertyData>	Valeurs générées et éditables pour ce slot.

12. Fonctionnement du parser Shader Graph

ShaderGraphCategoryParser est un utilitaire editor-only. Il sert à récupérer les catégories du Blackboard Shader Graph, car l'API Shader standard ne fournit pas directement cette information.

Entrée et sortie

Élément	Description
Entrée	UnityEngine.Object shaderGraphAsset, normalement un asset .shadergraph.
Sortie	Dictionary<string, string> où la clé est le Reference Name shader et la valeur est le nom de catégorie.

Principe

```
.shadergraph
├── CategoryData
│   ├── m_Name = "Global"
│   └── m_ChildObjectList = ids des propriétés
├── Vector1ShaderProperty
│   ├── m_ObjectId = "... "
│   └── m_OverrideReferenceName / m_DefaultReferenceName = "_GlobalAlpha"
├── ColorShaderProperty
└── ...
```

Le parser construit d'abord une table objectId → propertyReference, puis lit chaque CategoryData et associe les enfants de la catégorie au nom de catégorie.

```
Dictionary<string, string> objectIdToPropertyReference = new Dictionary<string, string>();
List<CategoryData> categories = new List<CategoryData>();

// 1. Lire les propriétés Shader Graph
// 2. Lire les catégories
// 3. Associer child.m_Id à une propriété
// 4. Retourner propertyReference -> categoryName
```

Pourquoi UnityEngine.Object ?

Les fichiers .shadergraph ne sont pas toujours importés comme TextAsset. La version actuelle utilise UnityEngine.Object pour référencer l'asset et lit le contenu avec File.ReadAllText à partir du chemin AssetDatabase.

Limitation assumée

Fragilité — Le parser dépend du format interne des fichiers .shadergraph. Si Unity change ce format, la génération des propriétés continuera de fonctionner, mais les catégories pourront tomber dans Other.

13. Workflow Shader Graph recommandé

Pour obtenir une UI propre dans MPB Generator, le Shader Graph doit être organisé dès le départ. Le Blackboard doit contenir des catégories lisibles et des Reference Names stables.

Élément	Recommandation	Exemple
Display Name	Nom lisible pour l'utilisateur	Blood Intensity
Reference Name	Nom stable utilisé côté code	_BloodIntensity
Catégorie	Nom court et fonctionnel	Global, Blood, Noise, Advanced
Bool / Toggle	Attribut Toggle ou nom contenant toggle/bool	_ToggleDissolve, _IsBoolDebug

Organisation conseillée

```

Shader Graph Blackboard
├── Global
│   ├── _GlobalAlpha
│   ├── _GlobalNoiseStrength
│   └── _GlobalDissolveSoftness
├── Blood Layers
│   ├── _DryColor
│   ├── _MidColor
│   └── _FreshColor
├── Noise
│   ├── _NoiseScale
│   ├── _NoiseOffset
│   └── _NoiseStrength
└── Advanced
    ├── _DebugMode
    └── _UseVertexColor
  
```

Après modification du Shader Graph

- Si une propriété est renommée, cliquer à nouveau sur Generate.
- Si une catégorie est modifiée, cliquer à nouveau sur Generate.
- Si le Reference Name change, les anciennes valeurs MPB peuvent ne plus affecter le shader.
- Conserver les Reference Names stables une fois que des scènes utilisent l'outil.

14. Gestion des types de propriétés shader

Chaque ShaderPropertyType est converti en champ UI puis en méthode MaterialPropertyBlock.

ShaderPropertyType	Champ UI	Valeur stockée	Méthode MPB
Color	ColorField	ColorValue	SetColor
Vector	Vector4Field	VectorValue	SetVector
Float	FloatField ou Toggle	FloatValue	SetFloat
Range	Slider	FloatValue + RangeMin/RangeMax	SetFloat
Texture	ObjectField Texture	TextureValue	SetTexture
Int	IntField	IntValue	SetInt

Color HDR

Si ShaderPropertyFlags.HDR est présent, l'Editor peut ouvrir un ColorField compatible HDR. Cela permet de gérer les propriétés d'émission ou les couleurs dont l'intensité dépasse 1.

Range

Les propriétés Range sont affichées en slider avec les limites récupérées via shader.GetPropertyRangeLimits. Ces limites sont stockées dans ShaderPropertyData pour éviter de les recalculer pendant l'affichage.

Texture

Les textures sont appliquées uniquement si TextureValue n'est pas null. Cela évite d'écraser involontairement une texture du Material avec une valeur vide. Une option future pourrait permettre de forcer SetTexture avec null.

15. Gestion des booléens et toggles

Unity ne renvoie pas de `ShaderPropertyType.Bool` pour ce workflow. Un bool shader est généralement représenté par un `Float` ou un `Int` à 0/1. L'outil utilise donc un champ `IsToggle` pour décider si un `Float` doit être dessiné comme une checkbox.

Détection actuelle

La détection doit rester prudente. Elle se base principalement sur l'attribut `Toggle`, puis sur un fallback par nom contenant `toggle` ou `bool`. Les mots génériques comme `Use` ou `Enable` ne sont pas recommandés pour la détection automatique, car ils peuvent provoquer des faux positifs.

```
private static bool IsToggleProperty(Shader shader, int propertyIndex)
{
    string[] attributes = shader.GetPropertyAttributes(propertyIndex);

    foreach (string attribute in attributes)
    {
        if (attribute.Contains("Toggle"))
            return true;
    }

    string propertyName = shader.GetPropertyName(propertyIndex).ToLowerInvariant();
    string displayName = shader.GetPropertyDescription(propertyIndex).ToLowerInvariant();

    if (propertyName.Contains("toggle") ||
        propertyName.Contains("bool") ||
        displayName.Contains("toggle") ||
        displayName.Contains("bool"))
    {
        return true;
    }

    return false;
}
```

Affichage

```
if (property.IsToggle)
{
    bool value = property.FloatValue > 0.5f;
    value = EditorGUILayout.Toggle(label, value);
    property.FloatValue = value ? 1f : 0f;
}
else
{
    property.FloatValue = EditorGUILayout.FloatField(label, property.FloatValue);
}
```

Conseil Shader Graph — Pour une propriété booléenne, ajoute un attribut `Toggle` quand c'est possible. C'est plus fiable qu'une détection basée sur le nom.

16. Gestion des catégories

La catégorie d'une propriété est stockée dans `ShaderPropertyData.Category`. Le runtime ne décide pas de la catégorie : elle est calculée au moment de la génération, puis l'Editor se contente de regrouper les propriétés ayant le même nom de catégorie.

```
string category = "Other";

if (shaderGraphCategories.TryGetValue(propertyName, out string parsedCategory))
{
    category = parsedCategory;
}

ShaderPropertyData data = new ShaderPropertyData
{
    Name = propertyName,
    Category = category,
    Type = propertyType
};
```

Côté UI

L'Editor construit une liste de catégories uniques en parcourant `Properties`, puis dessine une section par catégorie.

```
List<string> categories = GetCategories(script);

foreach (string category in categories)
{
    DrawCategory(category, script);
}
```

Catégorie Other

Other est le fallback. Une propriété arrive dans Other si le `ShaderGraphAsset` est null, si le fichier n'est pas un `.shadergraph`, si le parser échoue ou si la propriété n'est attachée à aucune catégorie reconnue.

Ordre des catégories

L'ordre actuel suit la première apparition des catégories dans la liste des propriétés générées. Une version future pourrait récupérer l'ordre exact des `CategoryData` ou appliquer une priorité configurée.

17. Gestion des material slots

Le MaterialPropertyBlock s'applique au Renderer, pas au Material. C'est pour cette raison que MaterialIndex est nécessaire dès qu'un Renderer possède plusieurs Materials, ou quand le même Material est utilisé dans plusieurs slots.

Cas	SourceMaterial suffit ?	MaterialIndex nécessaire ?
Un seul Material sur le Renderer	Oui, généralement	Possible mais pas indispensable
Plusieurs Materials différents	Partiellement	Oui pour cibler le bon slot
Même Material utilisé dans plusieurs slots	Non	Oui absolument

Dropdown de slot

Le dropdown lit renderer.sharedMaterials et affiche Element i - MaterialName. Il synchronise ensuite MaterialIndex et SourceMaterial.

Réorganisation des slots

Dans MPBGeneratorList, les boutons de réorganisation ne changent pas MaterialIndex. Ils changent uniquement l'ordre d'affichage de la liste Slots pour rendre l'Inspector plus lisible.

```

if (GUILayout.Button("\u25B2", GUILayout.Width(25)))
{
    SwapSlots(script, slotListIndex, slotListIndex - 1);
}

if (GUILayout.Button("\u25BC", GUILayout.Width(25)))
{
    SwapSlots(script, slotListIndex, slotListIndex + 1);
}

```

18. Samples et exemple BloodHandSplatter

Le dossier Samples~ contient un exemple BloodHandSplatter. Il sert à montrer le workflow complet : Shader Graph organisé, Material source, textures, subgraph et scène de démonstration.

Asset	Rôle
BloodHandSplatter.mat	Material source utilisé par le generator.
BloodHandSplatterShaderGraph.shadergraph	Shader Graph principal avec propriétés exposées et catégories.
MakeLayer.shadersubgraph	Subgraph de composition des couches de sang.
blood_organic_grunge_noise_1024.png	Texture de noise organique.
blood_organic_grunge_noise_1024_inverted.png	Variante inversée de la texture de noise.
BloodHandSplatterExample.unity	Scène de démonstration.

Workflow recommandé pour le sample

31. Importer le sample depuis le Package Manager.
32. Ouvrir BloodHandSplatterExample.unity.
33. Sélectionner un objet de démonstration avec Renderer.
34. Ajouter MPBGenerator ou MPBGeneratorList.
35. Choisir le material slot BloodHandSplatter.
36. Cliquer Generate.
37. Modifier les valeurs de couleurs, alpha, noise, smoothness ou offsets.
38. Vérifier que l'objet réagit sans modifier BloodHandSplatter.mat.

Contrôle de smoothness

Pour un shader de sang, un pattern simple consiste à multiplier le canal de smoothness par un float pilotable par MPB, puis à saturer le résultat.

```
SmoothnessMapChannel
→ Multiply(SmoothnessStrength)
→ Saturate
→ Smoothness
```

19. Bonnes pratiques de production

- Ne pas modifier le Material asset depuis le custom Inspector MPB.
- Toujours régénérer les propriétés après un changement de Shader Graph.
- Utiliser des Reference Names stables.
- Éviter de renommer les propriétés quand des scènes utilisent déjà le tool.
- Laisser Include Hidden Properties désactivé sauf debug spécifique.
- Utiliser MPBGeneratorList quand un Renderer possède plusieurs Materials.
- Tester Clear MPB pour vérifier qu'aucun override ne reste.
- Conserver un sample minimal qui démontre le workflow complet.
- Documenter les versions Unity testées dans README.md.

Workflow recommandé pour un decal de sang

39. Créer les propriétés du Shader Graph : couleurs, alpha, noise, smoothness, offsets.
40. Organiser le Blackboard en catégories : Global, Blood Layers, Noise, Advanced.
41. Créer un Material source propre avec des valeurs par défaut.
42. Ajouter MPBGenerator sur le decal si un seul material slot est utilisé.
43. Ajouter MPBGeneratorList si plusieurs materials doivent être pilotés.
44. Générer les propriétés.
45. Ajuster les valeurs par instance.
46. Tester en Play Mode que les valeurs restent indépendantes entre objets.

20. Performance et limites

Avantages performance

L'intérêt du MPB est d'éviter la duplication de Materials pour des variations simples par objet. Le même Material peut être partagé par plusieurs Renderers, tandis que chaque Renderer possède des valeurs différentes via son MPB.

Ce que le tool ne garantit pas

- Il ne garantit pas automatiquement le GPU instancing.
- Il ne rend pas une propriété active si elle n'est pas utilisée dans le Shader Graph.
- Il ne contourne pas les limitations de Shader Graph.
- Il ne garantit pas la stabilité du format .shadergraph entre versions Unity.
- Il ne remplace pas les drawers internes complexes du Material Inspector Unity.

Coût Editor

Le parsing et la génération se produisent principalement quand l'utilisateur clique sur Generate. L'application du MPB est légère. Si une scène contient beaucoup de slots et textures, il peut être intéressant d'appliquer uniquement le slot modifié plutôt que ApplyAll à chaque changement GUI.

22. Troubleshooting détaillé

Problème	Cause probable	Solution
Toutes les propriétés sont dans Other	ShaderGraphAsset null ou parser incapable de trouver les catégories	Vérifier le champ Shader Graph, cliquer Generate, vérifier le format .shadergraph.
Shader Graph non assigné automatiquement	AssetDatabase.GetAssetPath(shader) ne retourne pas un .shadergraph	Assigner manuellement le .shadergraph ou adapter AutoAssignShaderGraphAsset.
Erreur duplicate ShaderGraphCategoryParser	La classe existe dans plusieurs fichiers	Garder une seule classe dans Editor/ShaderGraph/ShaderGraphCategoryParser.cs.
Les toggles restent en float	Pas d'attribut Toggle et nom sans toggle/bool	Ajouter un attribut Toggle ou renommer la propriété.
Le MPB ne modifie rien	Mauvais material slot, propriété non utilisée ou mauvais Reference Name	Vérifier MaterialIndex, Reference Name et branchement Shader Graph.
Le material asset change	Le Material est modifié directement ou sharedMaterial est assigné au mauvais endroit	Ne pas éditer le Material Inspector ; vérifier la logique d'Init.
Erreur UnityEditor en build	Un script Runtime référence UnityEditor hors #if	Déplacer dans Editor ou protéger avec #if UNITY_EDITOR.
Dropdown material vide	Renderer sans Materials	Assigner au moins un Material au Render.
Flèches ne compilent pas	Encodage source incompatible	Utiliser \u25B2 et \u25BC au lieu des caractères copiés.
Texture ne s'applique pas	TextureValue null ou shader n'utilise pas la propriété	Assigner une texture et vérifier le Reference Name.

Checklist de debug rapide

47. Regarder la Console Unity et corriger la première erreur.
48. Vérifier que les scripts Editor sont bien dans un dossier Editor.
49. Vérifier qu'il n'existe qu'un seul ShaderGraphCategoryParser.
50. Vérifier que le Renderer possède au moins un Material.
51. Vérifier que MaterialIndex cible le bon Element.
52. Logger AssetDatabase.GetAssetPath(SourceMaterial.shader).
53. Vérifier que le chemin finit par .shadergraph.
54. Cliquer Generate après toute modification du Shader Graph.
55. Vérifier que Properties.Count est supérieur à 0.
56. Vérifier que property.Name correspond au Reference Name dans Shader Graph.

Logs utiles

```

Debug.Log($"Shader = {shader.name}");
Debug.Log($"Shader Path = {shaderPath}");
Debug.Log($"Properties count = {script.Properties.Count}");
Debug.Log($"Property: {property.Name} | Category: {property.Category}");

```

23. FAQ

Pourquoi ne pas utiliser directement MaterialEditor ?

MaterialEditor dessine l'Inspector du Material et modifie le Material asset. MPB Generator doit modifier des valeurs sérialisées dans un composant puis les appliquer à un MaterialPropertyBlock.

Pourquoi parser le .shadergraph ?

Parce que l'API Shader standard ne donne pas directement les catégories du Blackboard Shader Graph.

Est-ce que le tool fonctionne avec des shaders non Shader Graph ?

Oui pour générer et appliquer les propriétés. Les catégories tomberont simplement dans Other si aucun .shadergraph n'est disponible.

MPBGeneratorList remplace-t-il MPBGenerator ?

Non. MPBGenerator reste plus simple pour un seul slot. MPBGeneratorList est utile pour plusieurs material slots.

Pourquoi les bools sont des floats ?

C'est courant côté shader Unity. Un toggle est souvent stocké comme 0 ou 1.

Les valeurs sont-elles sauvegardées dans la scène ?

Oui. Properties et Slots sont sérialisés dans le composant MonoBehaviour.

Clear MPB efface-t-il Properties ?

Non. Clear retire l'override du Renderer mais conserve les données sérialisées.

Peut-on créer des presets ?

Oui, une future extension pourrait utiliser des ScriptableObjects pour sauvegarder et réappliquer des groupes de valeurs.

Pourquoi le parser est editor-only ?

Parce qu'il utilise AssetDatabase et lit le fichier .shadergraph, ce qui n'existe pas dans un build player.

Dois-je inclure les hidden properties ?

Seulement pour debug ou cas très spécifique. Par défaut, il vaut mieux les masquer.

Annexe A — Glossaire

Terme	Définition
MPB	MaterialPropertyBlock, bloc de valeurs shader appliqué à un Renderer.
Renderer	Composant Unity qui dessine un objet et reçoit les Materials.
Material slot	Index d'un Material dans <code>renderer.sharedMaterials</code> .
Shader Graph Asset	Fichier <code>.shadergraph</code> source utilisé pour retrouver les catégories.
Reference Name	Nom technique de propriété shader utilisé côté code.
Display Name	Nom affiché à l'utilisateur dans l'Inspector.
Category	Groupe d'affichage provenant du Blackboard Shader Graph.
Hidden Property	Propriété shader marquée <code>HideInInspector</code> .
Toggle	Booléen shader généralement stocké comme float 0/1.
Sample	Exemple importable depuis <code>Samples~</code> dans le Package Manager.

Annexe B — Snippets utiles

Auto-assign Shader Graph

```
Shader shader = sourceMaterial.shader;
string shaderPath = AssetDatabase.GetAssetPath(shader);

if (!string.IsNullOrEmpty(shaderPath) && shaderPath.EndsWith(".shadergraph"))
{
    Object shaderGraphAsset = AssetDatabase.LoadMainAssetAtPath(shaderPath);
}
```

Apply per material slot

```
MaterialPropertyBlock mpb = new MaterialPropertyBlock();

// Remplir le MPB

renderer.SetPropertyBlock(mpb, materialIndex);
```

Swap de slots

```
private static void SwapSlots(MPBGeneratorList script, int a, int b)
{
    Undo.RecordObject(script, "Reorder MPB Slots");

    MPBSlot temp = script.Slots[a];
    script.Slots[a] = script.Slots[b];
    script.Slots[b] = temp;

    EditorUtility.SetDirty(script);
}
```

Boutons compatibles encodage

```
GUILayout.Button("\u25B2", GUILayout.Width(25)); // triangle haut
GUILayout.Button("\u25BC", GUILayout.Width(25)); // triangle bas
```